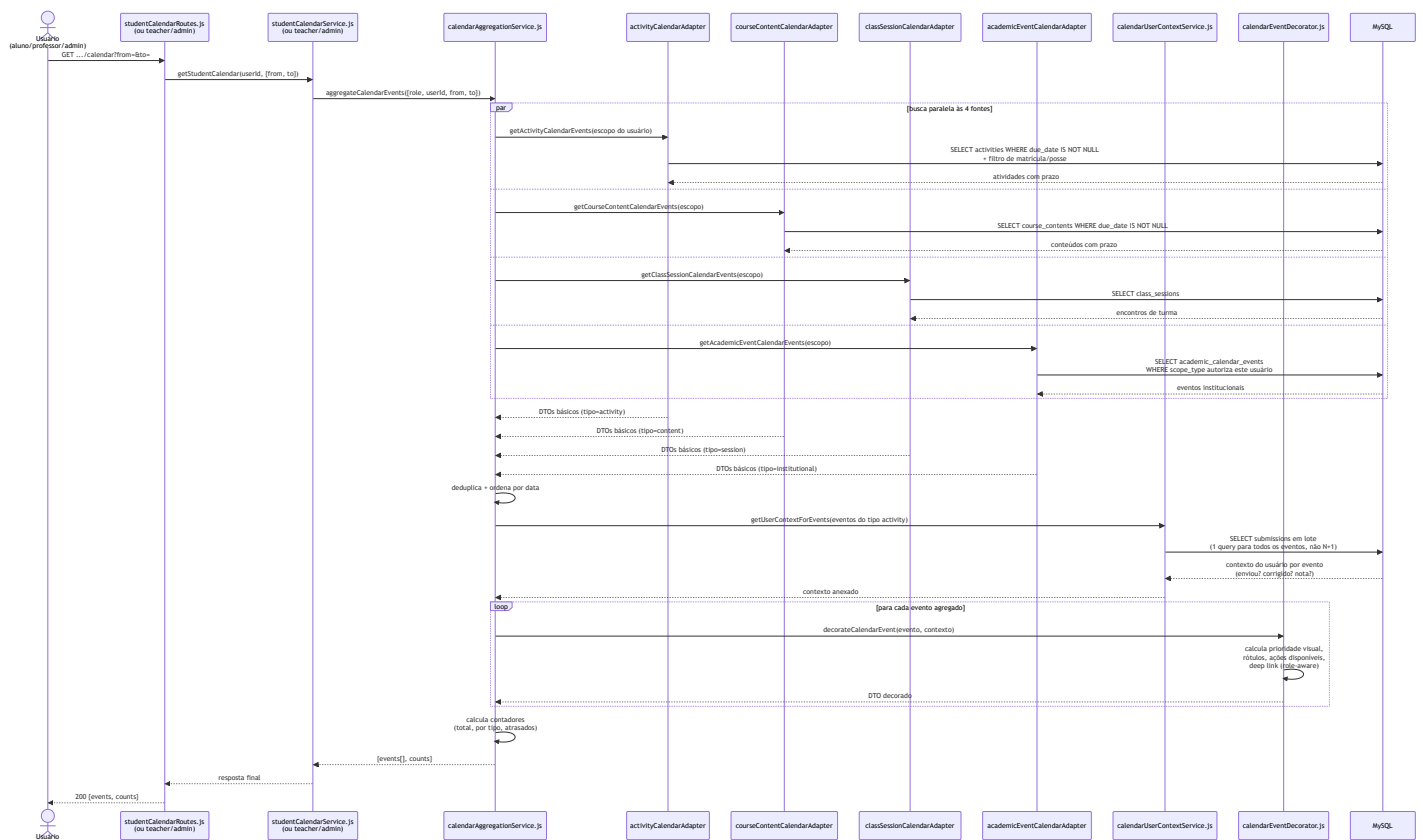


Diagrama — Fluxo do Calendário Acadêmico

O calendário é um agregador somente-leitura (ver ADR-001, Decisão 2). Este diagrama mostra exatamente como uma requisição de calendário combina 4 fontes de dados em uma resposta única. Nomes reais: `calendarAggregationService.js` , `calendarEventDecorator.js` , `calendarUserContextService.js` , os 4 adapters em `services/calendar/adapters/` .



Que problema este diagrama esclarece

A pergunta mais comum sobre este módulo é "o calendário guarda os prazos, ou só mostra?" — e a resposta correta (só mostra) só fica realmente convincente quando se vê, passo a passo, que 4 fontes de dados independentes são consultadas a cada requisição, com zero escrita em `academic_calendar_events` por parte de atividades/conteúdos/sessões. Este diagrama é a prova visual de que remover o módulo de calendário não perde nenhum dado — os dados nunca estiveram lá.

Como interpretar os elementos

- O bloco `par` (busca paralela) é literal: os 4 adapters são chamados com `Promise.all`, não sequencialmente — o diagrama reflete a implementação real, não uma simplificação didática.
- `calendarUserContextService.js` busca contexto **em lote** para todos os eventos de atividade de uma vez (uma query), explicitamente citado no diagrama como "não N+1" porque essa é uma decisão de performance deliberada, não incidental — buscar o contexto evento por evento seria o caminho ingênuo e mais lento.
- O `loop` de decoração roda por evento porque `calendarEventDecorator.js` é uma função pura por item (prioridade, rótulos, deep link dependem dos dados daquele evento específico), diferente da busca de contexto, que é deliberadamente em lote.
- O mesmo router/service genérico (`studentCalendarRoutes` / `studentCalendarService`) é reutilizado nos três papéis — o diagrama usa comentários "(ou teacher/admin)" para deixar explícito que a estrutura é idêntica para os três, só o escopo de autorização dentro de cada adapter muda.

Que decisões arquiteturais este diagrama evidencia

- **Autorização de escopo acontece dentro de cada adapter, não em uma camada central:** cada um dos 4 adapters recebe `role` e `userId` e decide sozinho quais linhas retornar (matrícula do aluno, posse do professor, escopo institucional). Não existe um filtro pós-agregação que remova eventos não autorizados — a autorização é aplicada na origem, por design, para nunca buscar dado que o usuário não deveria ver.
- **Deep link é role-aware dentro do decorator:** o mesmo evento de atividade gera um link diferente para aluno (`/aluno/atividades/:id`) e professor (`/professor/atividades/:id/envios`) — essa lógica foi originalmente um bug (ignorava o papel) corrigido durante a implementação do módulo, e hoje é uma decisão explícita de design do decorator.
- **Zero escrita neste fluxo inteiro:** todo o diagrama, do início ao fim, é leitura. Isso não é uma coincidência de exemplo — o módulo de calendário não tem nenhuma rota de escrita além de `adminCalendarRoutes.js` para eventos institucionais, que é a única tabela própria do domínio.

Trade-offs que este fluxo representa

- **4 consultas paralelas em vez de uma consulta unificada:** mais simples de entender e evoluir cada fonte independentemente (adicionar um 5º tipo de evento é só adicionar um 5º adapter, sem migração de dados), em troca de mais round-trips ao banco por requisição do que uma tabela de calendário única teria.

- **Contexto do usuário buscado em lote, mas ainda uma query extra:** evita N+1, mas ainda é uma query adicional além das 4 dos adapters — o trade-off aceito é performance previsível (5 queries paralelas/sequenciais bem definidas) em vez de tentar espremer tudo em menos round-trips com joins mais complexos entre fontes heterogêneas.
- **Sem cache:** cada requisição de calendário refaz a agregação inteira. Correto sempre (nunca mostra dado desatualizado), mas mais caro que servir uma versão cacheada — aceitável hoje dado o volume, um ponto a revisar se o calendário se tornar uma tela de alto tráfego.

Como este diagrama deve evoluir

Se um 5º tipo de evento agregável for adicionado (por exemplo, "vencimento de fatura" no calendário do aluno), ele deve seguir o mesmo padrão de adapter — adicionar um novo participante à seção par deste diagrama, nunca inserir lógica condicional dentro de um adapter existente para múltiplos tipos. Se um mecanismo de cache for introduzido, este diagrama precisa de uma nota explícita sobre onde ele se encaixa (antes ou depois da decoração) e sob que condições é invalidado.